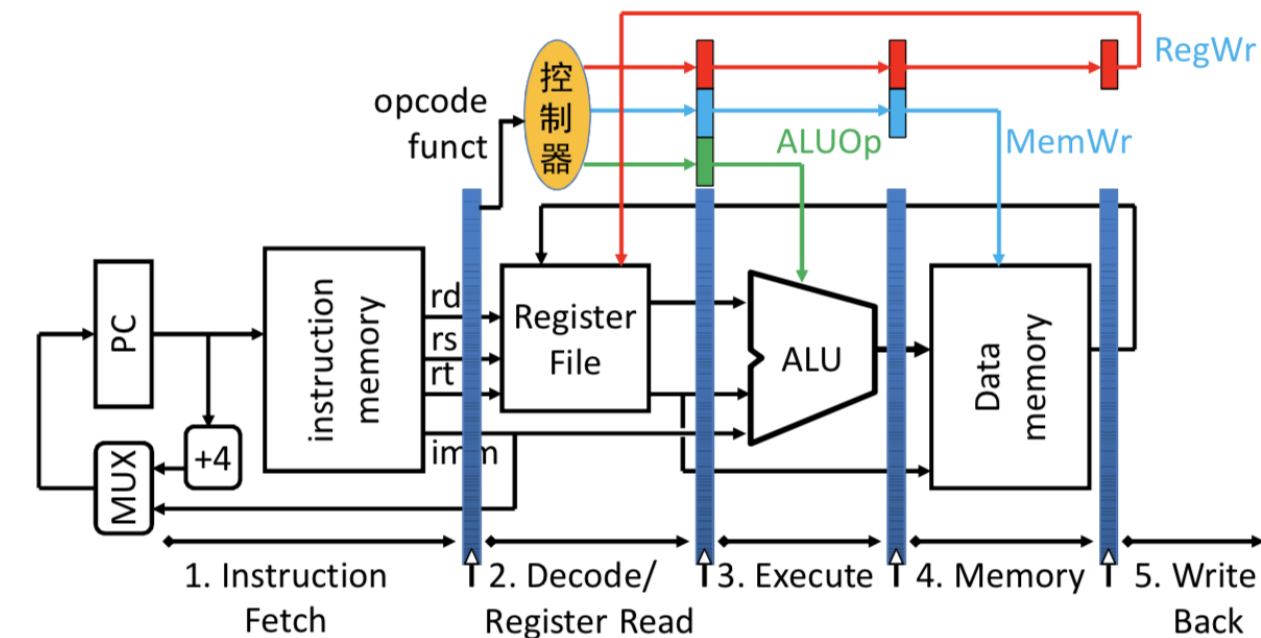


处理器

- 4.1 CPU概述
- 4.2 CPU设计的基本问题
- 4.3 MIPS模型机
- 4.4 单周期处理器设计
- 4.5 流水线技术
- 4.6 多周期处理器设计
- 4.7 流水线冒险
 - ◆ 4.7.1 结构冒险
 - ◆ 4.7.2 数据冒险
 - ◆ 4.7.3 控制冒险
- 4.8 异常处理
- 4.9 指令间的并行
- 4.10 流水线结构实例

多周期流水线CPU

- 流水线的控制信号 (CPU同时5条指令在运行)
 - 控制器：译码产生控制信号，与单周期完全相同
 - 控制信号流水寄存器：控制信号在寄存器中传递，直至不再需要



流水线冒险

- 流水线冒险 (Hazard)：流水线相近指令出现某些关联，下一个时钟周期不能执行下一条指令，指令流水线必须出现停顿。
 - ◆ 结构冒险：因缺乏硬件支持而导致指令不能在预定的时钟周期执行。
 - 要使用的部件正在忙（同一部件在不同阶段中被不同的指令同时使用）。例如，若系统只有一个存储器部件，就会带来结构冒险问题。
 - ◆ 数据冒险：指令执行所需的数据暂时不可用而导致指令不能在预定的时钟周期执行。
 - 数据冒险一般发生在相近指令访问同一个存储单元或寄存器时。后序指令需要等待前序指令执行完毕。
 - ◆ 控制冒险：也称为分支冒险 (branch hazard)，取到的指令不是所需要的而导致指令不能在预定的时钟周期执行。

内容

4.7.1 结构冒险

4.7.2 数据冒险

- ◆数据旁路

- ◆阻塞

4.7.3 控制冒险

- ◆阻塞

- ◆分支预测

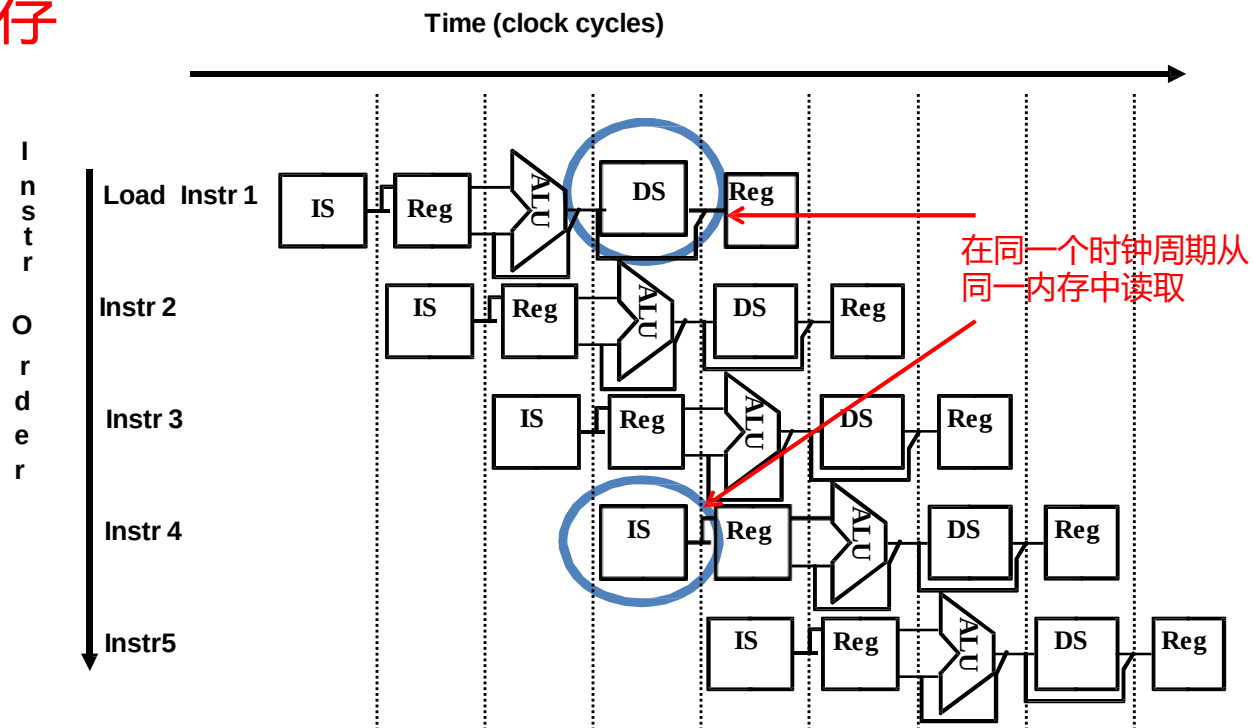
4.7.1 结构冒险

■所需资源冲突：

- ◆如果**MIPS**流水线仅有一个内存
 - 读或写的操作需要获取内存的权限
 - 取指需要等待一个周期：引入流水线气泡(**Bubble**)
- ◆因此，流水线的数据通路需要将指令和数据分别存储

结构冒险

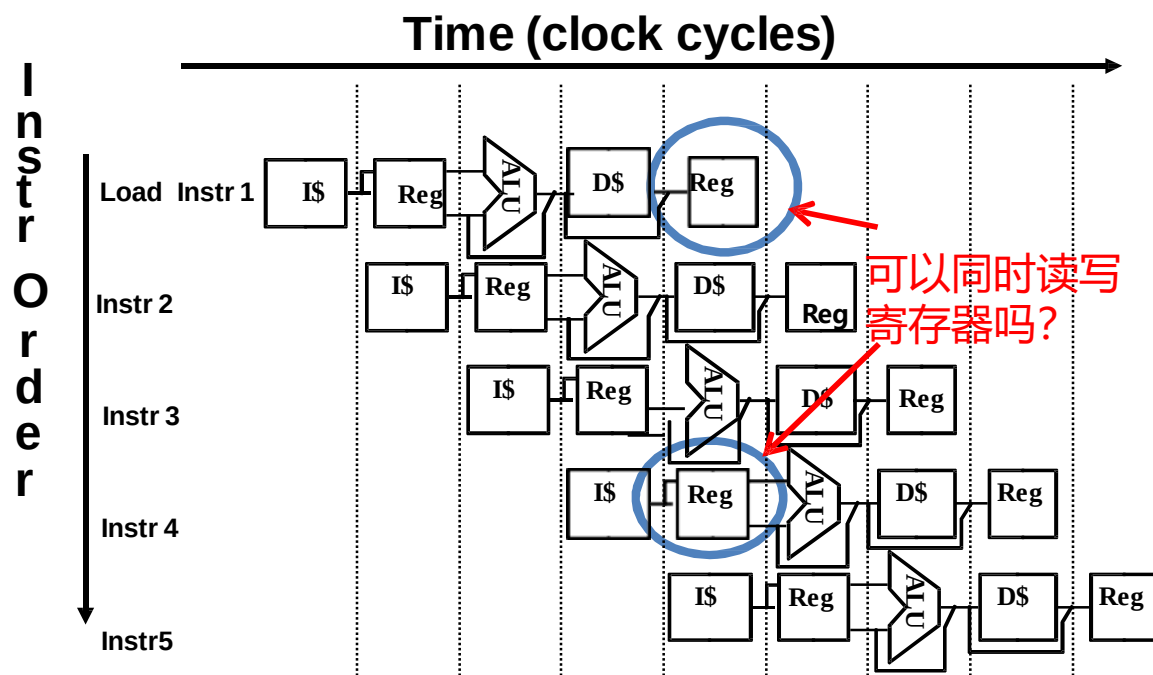
■ 内存



解决方案：使用独立的 IS 和 DS

结构冒险

■ #2: 寄存器

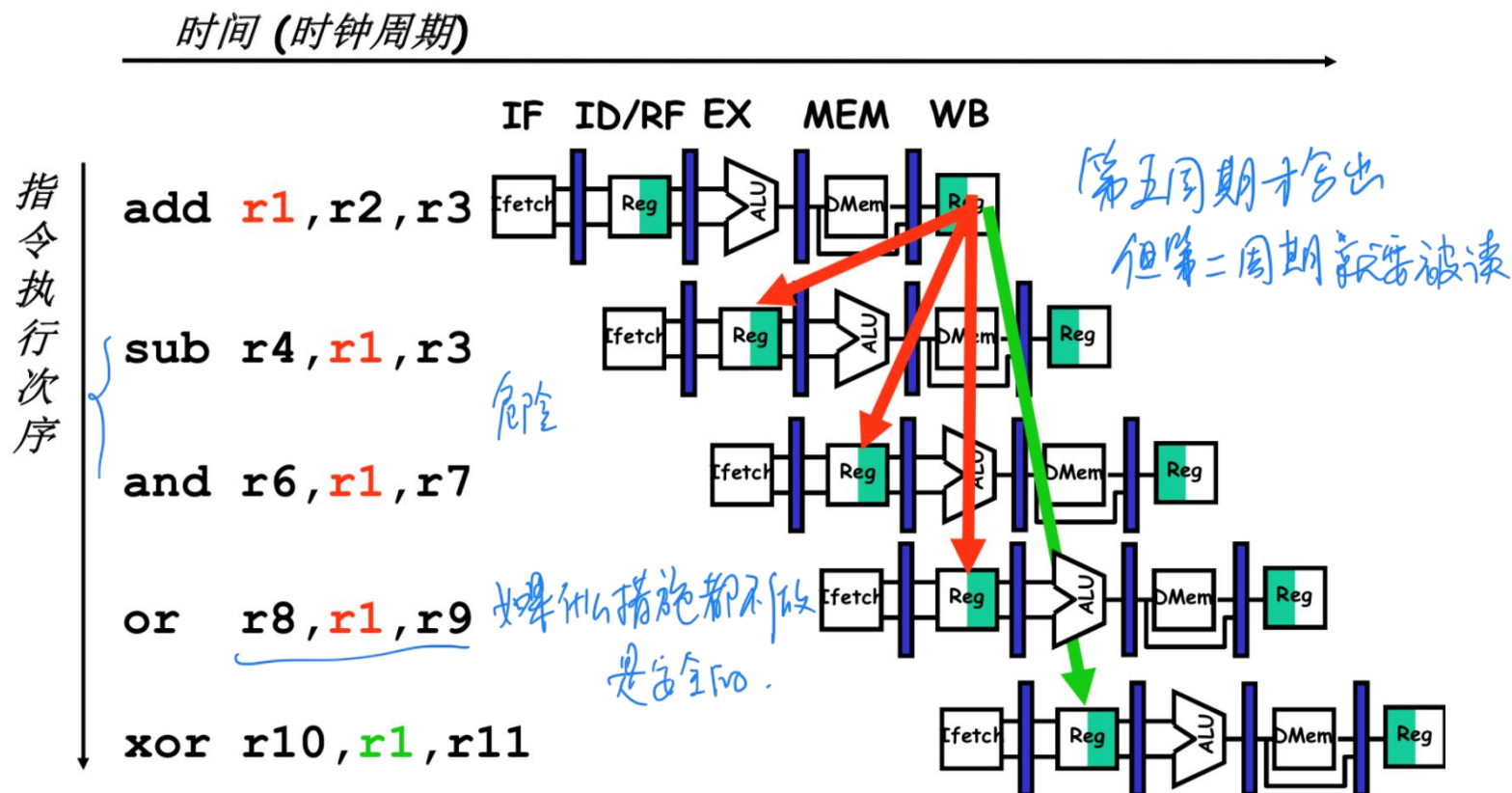


结构冒险

■ 寄存器

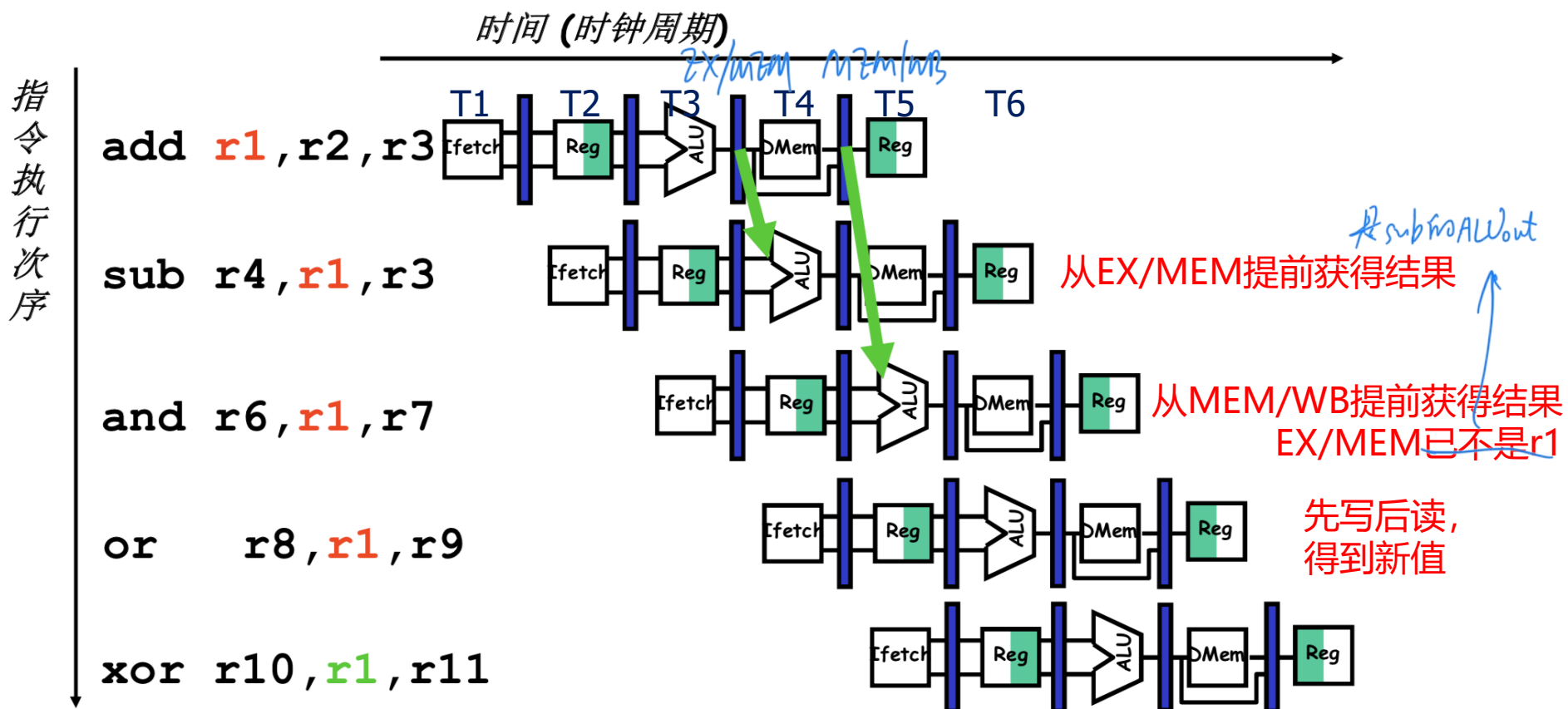
- ◆ 读操作类似组合逻辑，一直有效，读出新/旧数据取决于读的时机
 - 如果需要旧值，周期开始时读
 - 如果需要新值，后半个周期读（前半个周期写）
- ◆ 结论: 在同一个时钟周期，可以同时进行读寄存器和写寄存器的操作

4.7.2 数据冒险



数据冒险——旁路

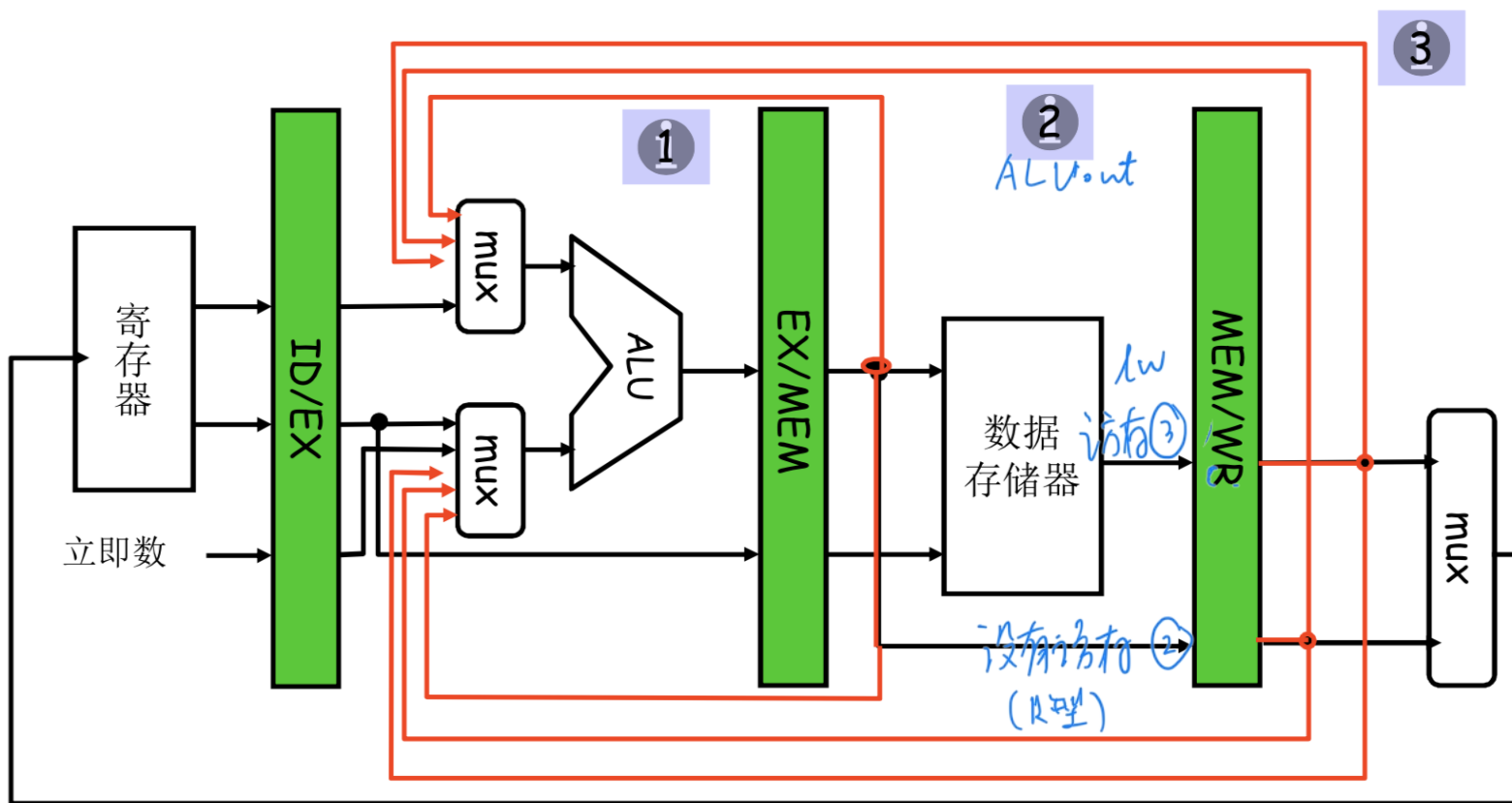
■ 解决策略：旁路(bypassing,转发forwarding)，提前给出结果



数据冒险——旁路

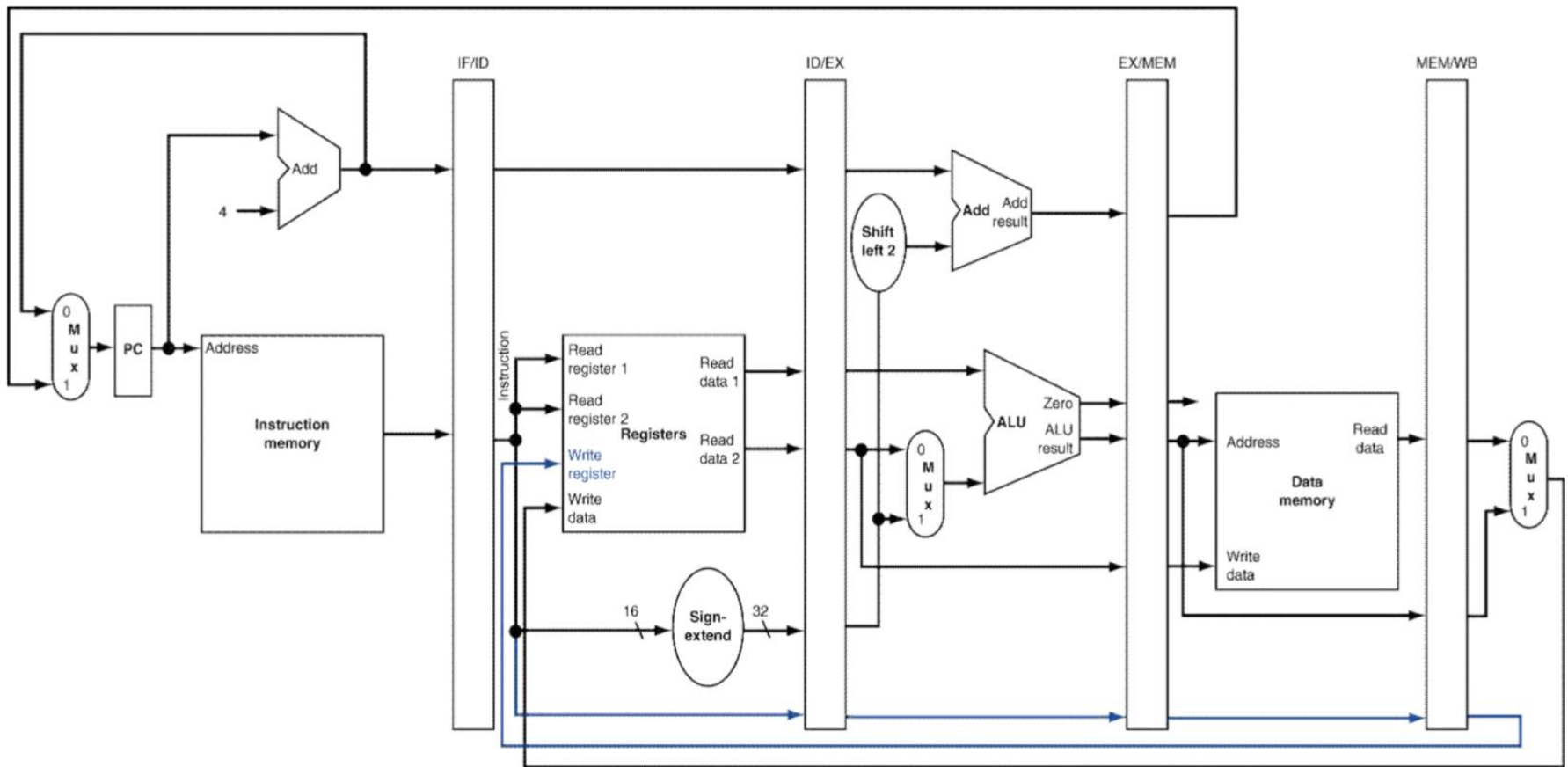
■ 调整硬件结构支持旁路

◆ EX/MEM -> ALU; MEM/WB -> ALU



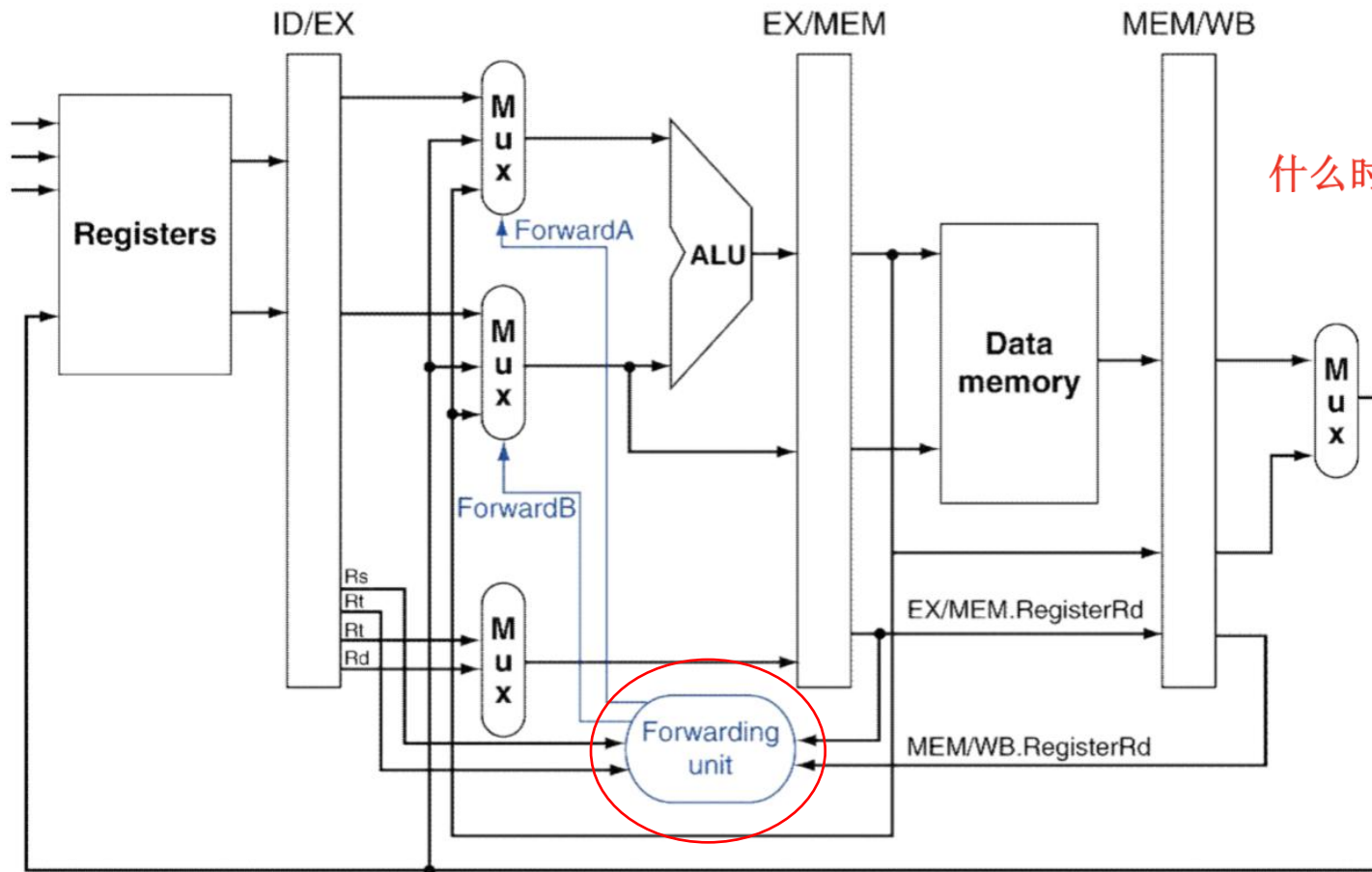
数据冒险——旁路

■ 数据通路如何修改？



数据冒险——旁路

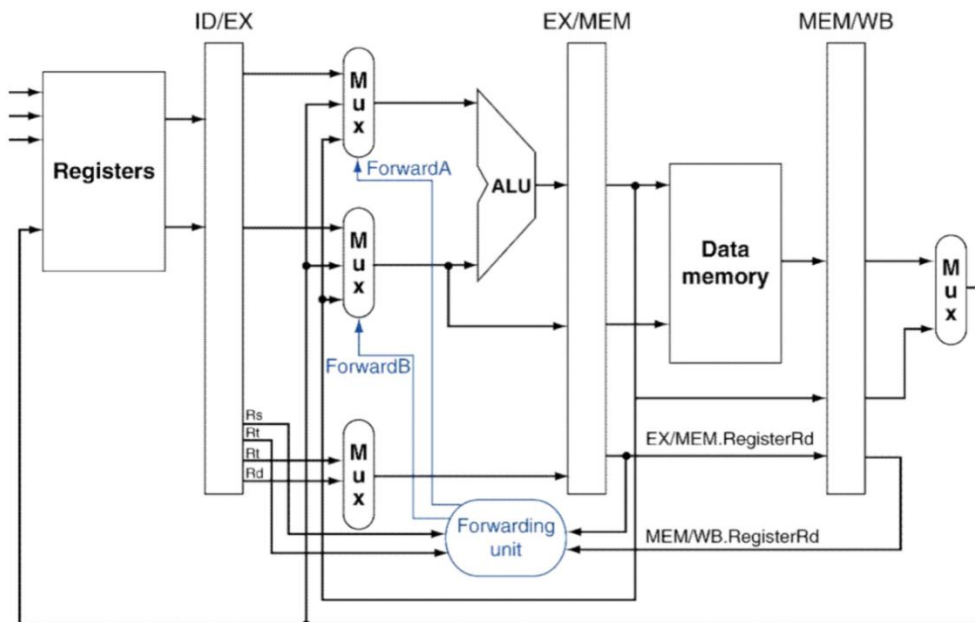
- 增加“旁路单元”，独立于控制器



什么时候需要转发？

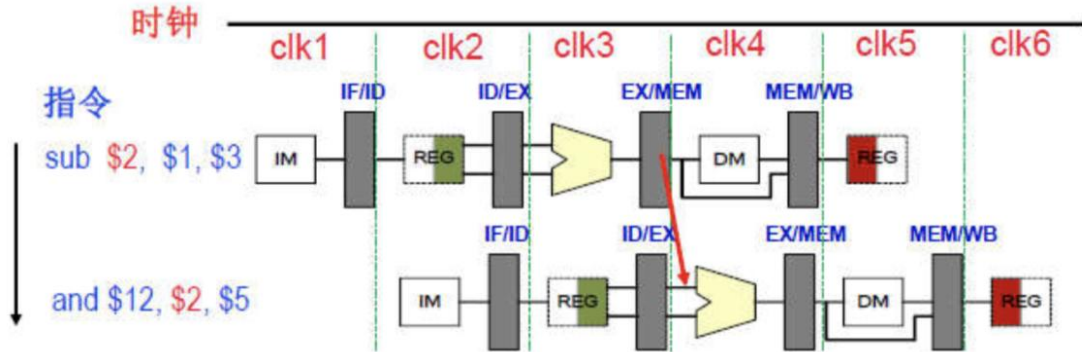
数据冒险——旁路

多选器控制	源	解释
ForwardA = 00	ID/EX	第一个 ALU 操作数来自寄存器堆
ForwardA = 10	EX/MEM	第一个 ALU 操作数由上一个 ALU 运算结果旁路获得
ForwardA = 01	MEM/WB	第一个 ALU 操作数从数据存储器或者前面的 ALU 结果中旁路获得
ForwardB = 00	ID/EX	第二个 ALU 操作数来自寄存器堆
ForwardB = 10	EX/MEM	第二个 ALU 操作数由上一个 ALU 运算结果旁路获得
ForwardB = 01	MEM/WB	第二个 ALU 操作数由数据存储器或者前面的 ALU 结果旁路获得



数据冒险——旁路

■什么情况选择EX/MEM?



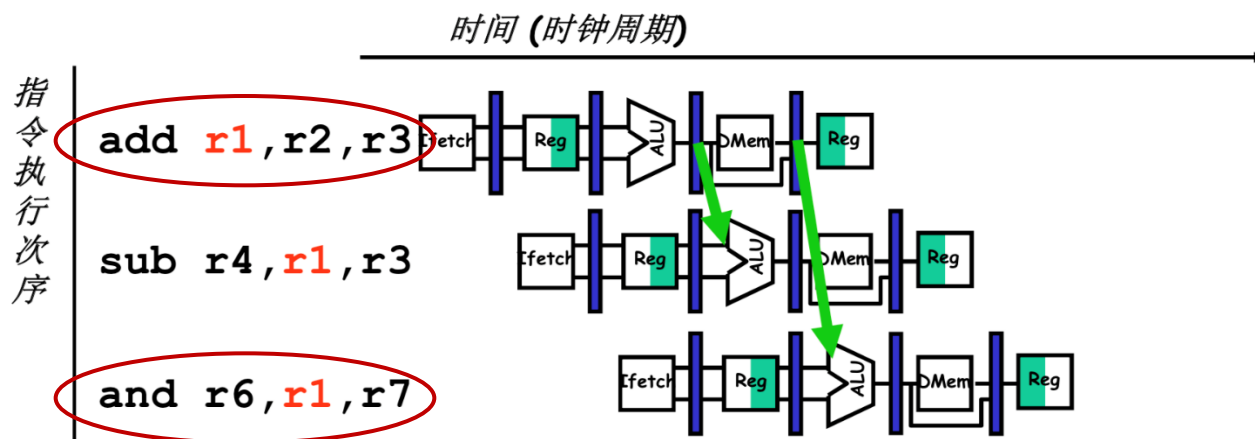
转发控制信号如何产生?

- `and`指令执行到EX级（`sub`指令执行到了MEM级）
- `sub`指令的`rd` = `and`指令的`rs`

if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0) 后一条指令要用的操作数是上一条指令要写的
and (EX/MEM.RegisterRd == ID/EX.RegisterRs)) ForwardA = 10
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd == ID/EX.RegisterRt)) ForwardB = 10

数据冒险——旁路

- 什么情况选择MEM/WB? (间隔一条指令)



if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
and (MEM/WB.RegisterRd == ID/EX.RegisterRs)) ForwardA = 01

if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
and (MEM/WB.RegisterRd == ID/EX.RegisterRt)) ForwardB = 01

数据冒险——旁路

■ 更复杂的情况

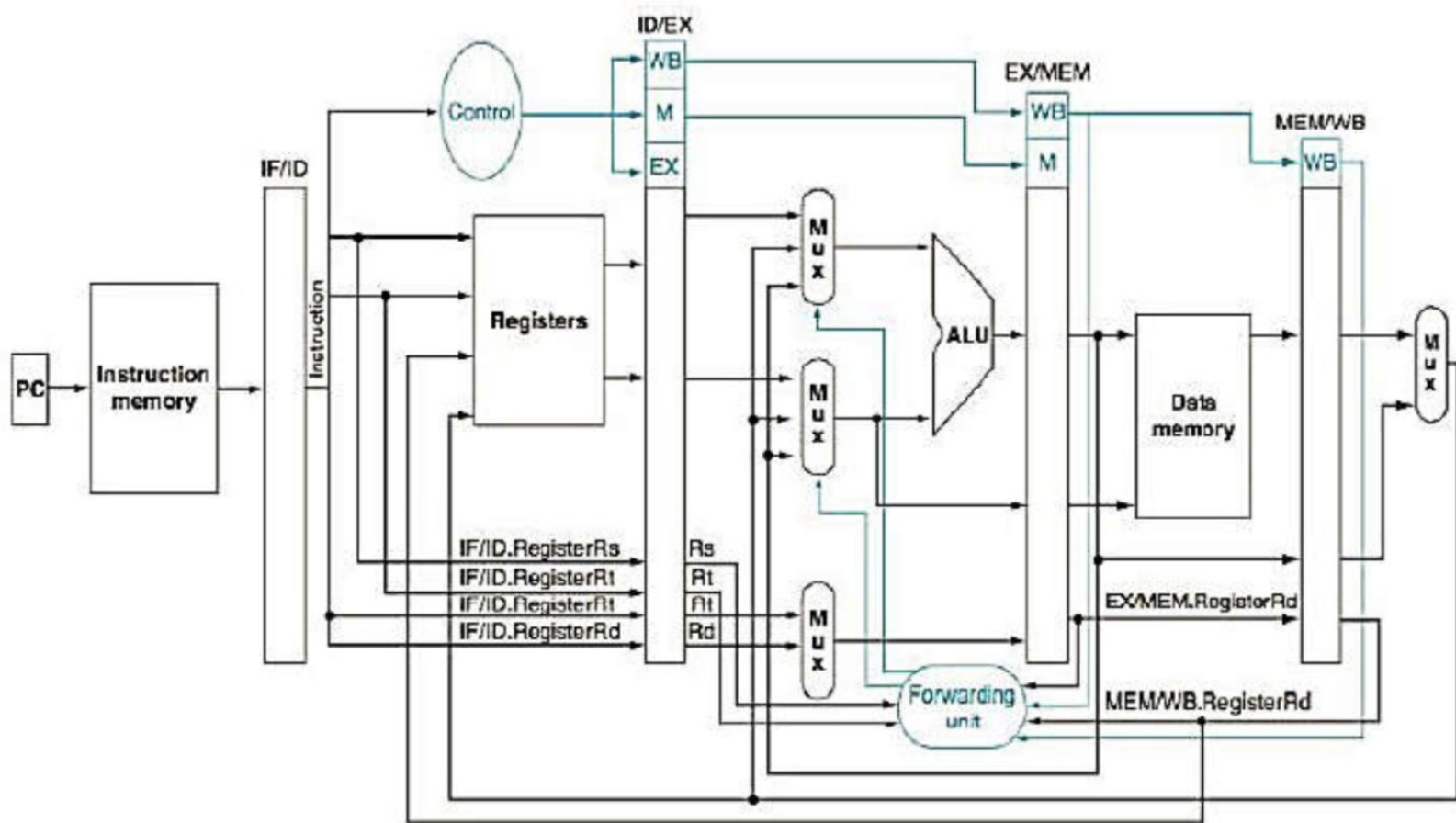
- ◆ EX/MEM和MEM/WB都满足条件，选择更新的数据来源：EX/MEM

if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
**and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd!=0)
and (EX/MEM.RegisterRd == ID/EX.RegisterRs))**
and (MEM/WB.RegisterRd==ID/EX.RegisterRs)) ForwardA = 01

not (EX/MEM 满足转发)

if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd != 0)
**and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd!=0)
and (EX/MEM.RegisterRd == ID/EX.RegisterRt))**
and (MEM/WB.RegisterRd==ID/EX.RegisterRt)) ForwardB = 01

数据冒险——旁路

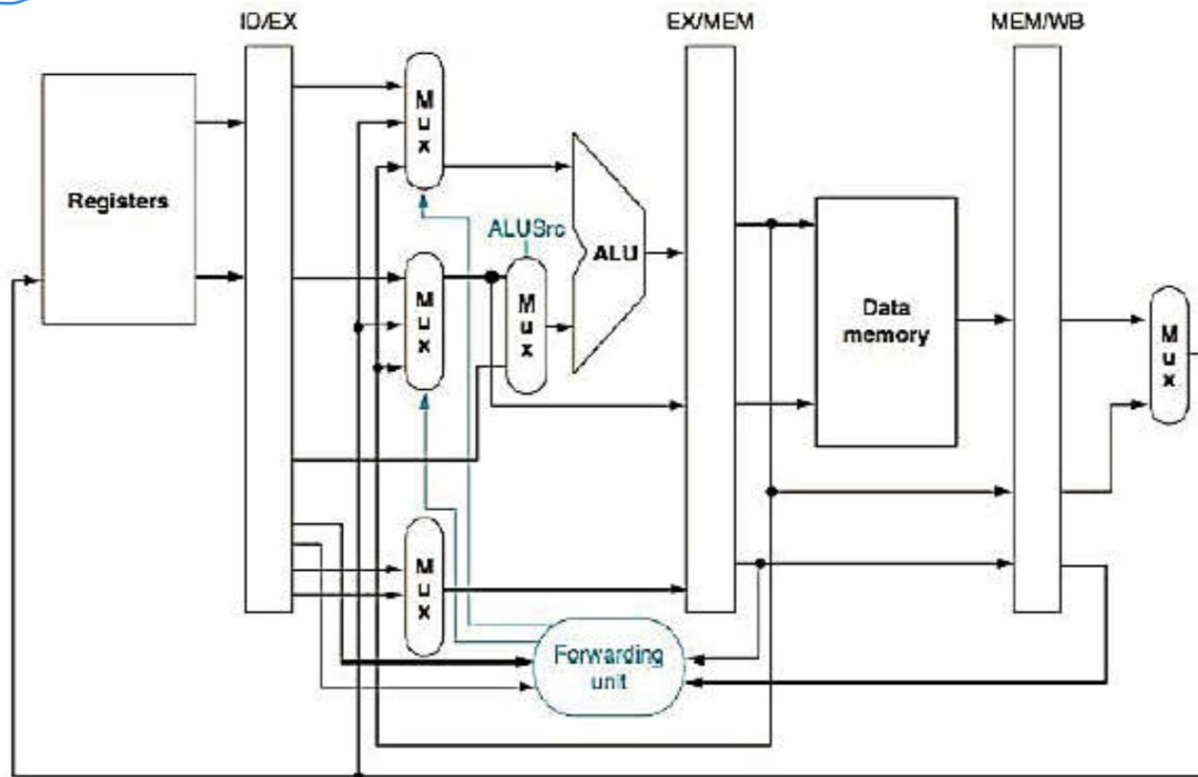


具备旁路转发功能的数据通路

数据冒险——旁路

■ 对指令中立即数送入ALU的情况

- ◆ 增加ALUSrc选择，多一个数据来源：指令立即数



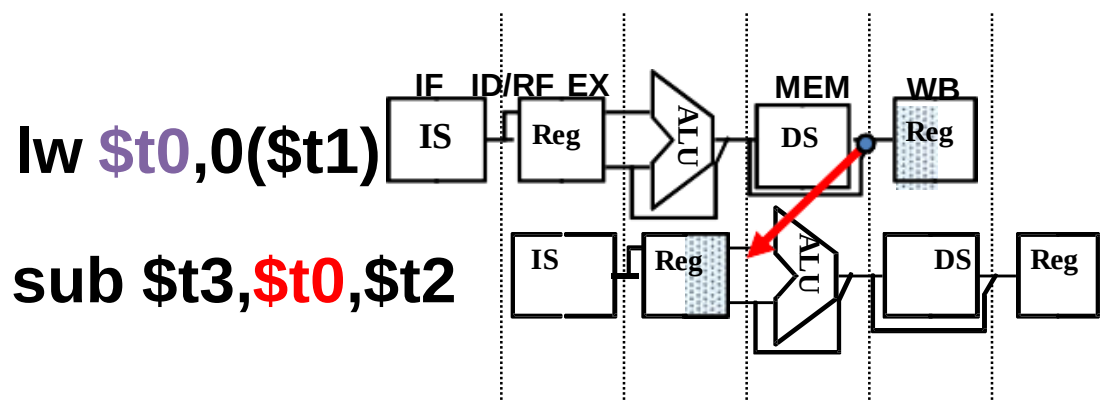
数据冒险——阻塞

■ 装载指令LW

- ◆ 在下图中，数据回传（时序出错）会导致数据冒险。

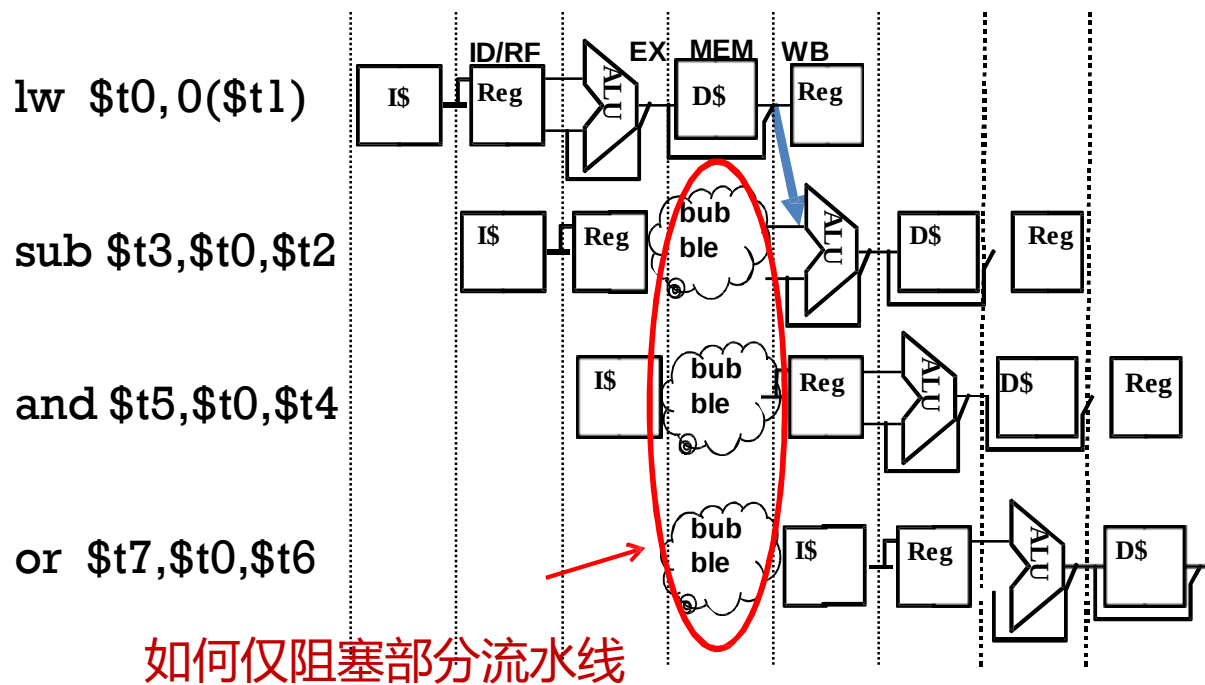
■ 数据旁路不能解决所有情况

- ◆ 一条指令试图读取一个由前一条装载指令读入到寄存器的数时，无法用旁路解决
- ◆ 必须阻塞依赖LW指令结果的指令
- ◆ 下图情况下，即使采用旁路也需要等待一个时钟周期



数据冒险——阻塞

■ 阻塞流水线



数据冒险——阻塞

- 阻塞等同于插入nop

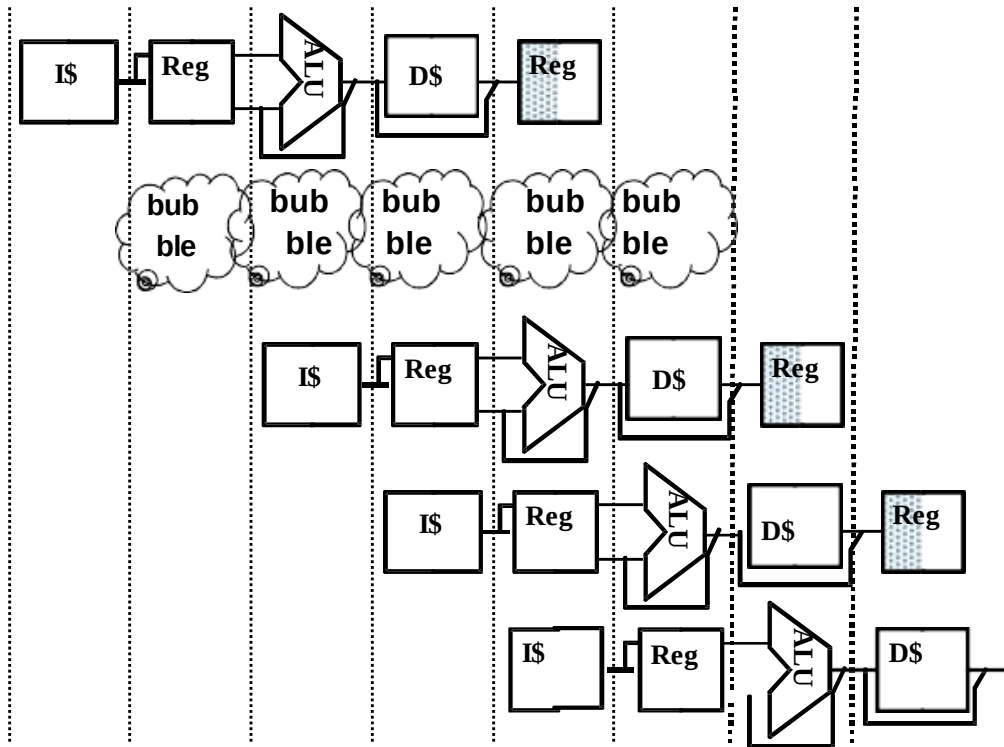
lw \$t0,0(\$t1)

nop

sub \$t3,\$t0,\$t2

and \$t5,\$t0,\$t4

or \$t7,\$t0,\$t6



数据冒险——阻塞

■ 装载指令导致的数据冒险：Clk1上升沿后

◆ 指令流

- lw进入IF/ID
- PC: 指向sub指令的地址
 - $PC \rightarrow PC + 4$
- IM: 输出sub指令

		IF级								ID级	EX级	MEM级	WB级
地址	指令	CLK	PC	IM	IF/ID	ID/EX	EX/MEM	MEM/WB	MEM/WB	RF			
0	lw \$t0, 0(\$t1)	↑ 1	0→4	lw→sub	lw								
4	sub \$t3, \$t0, \$t2												
8	and \$t5, \$t0, \$t4												
12	or \$t7, \$t0, \$t6												
16	add \$t1, \$t2, \$t3												

数据冒险——阻塞

■ 装载指令导致的数据冒险：Clk2上升沿后

◆ 指令流

➤ sub进入IF/ID寄存器；lw进入ID/EX寄存器

◆ 冲突分析：冲突出现

◆ 执行动作：设置控制信号，在clk3插入nop指令

➤ 冻结IF/ID：sub继续被保存

➤ 清除ID/EX：指令全为0，等价于插入NOP

➤ 禁止PC：防止PC继续计数，PC应保持为PC+4

		IF级								ID级	EX级	MEM级	WB级
地址	指令	CLK	PC	IM	IF/ID	ID/EX	EX/MEM	MEM/WB	RF				
0	lw \$t0, 0(\$t1)	↑ 1	0→4	lw→sub	lw								
4	sub \$t3, \$t0, \$t2	↑ 2	4→8	sub→and	sub	lw							
8	and \$t5, \$t0, \$t4												
12	or \$t7, \$t0, \$t6												
16	add \$t1, \$t2, \$t3												

数据冒险——阻塞

■ 装载指令导致的数据冒险：Clk3上升沿后

◆ 指令流

- sub进入ID/EX（冻结）；lw进入EX/MEM

◆ 冲突分析：

- 转发机制将在clk4时可以发挥作用

		IF级								ID级	EX级	MEM级	WB级
地址	指令	CLK	PC	IM	IF/ID	ID/EX	EX/MEM	MEM/WB	RF				
0	lw \$t0, 0(\$t1)	↑ 1	0→4	lw→sub	lw								
4	sub \$t3, \$t0, \$t2	↑ 2	4→8	sub→and	sub	lw							
8	and \$t5, \$t0, \$t4	↑ 3	8→8	and	sub	nop	lw						
12	or \$t7, \$t0, \$t6												
16	add \$t1, \$t2, \$t3												

数据冒险——阻塞

■ 装载指令导致的数据冒险：Clk4上升沿后

◆ 指令流

- lw: 结果存入MEM/WB。
- sub: 进入ID/EX。ALU的操作数可以从MEM/WB转发

◆ 执行动作

- 控制MUX, 使得MEM/WB输入到ALU

		IF级							
		IF级		ID级	EX级	MEM级	WB级		
地址	指令	CLK	PC	IM	IF/ID	ID/EX	EX/MEM	MEM/WB	RF
0	lw \$t0, 0(\$t1)	↑ 1	0→4	lw→sub	lw				
4	sub \$t3, \$t0, \$t2	↑ 2	4→8	sub→and	sub	lw			
8	and \$t5, \$t0, \$t4	↑ 3	8→8	and	sub	nop	lw		
12	or \$t7, \$t0, \$t6	↑ 4	8→12	and→or	and	sub	nop	lw结果	
16	add \$t1, \$t2, \$t3								

数据冒险——阻塞

■ 装载指令导致的数据冒险：Clk5上升沿后

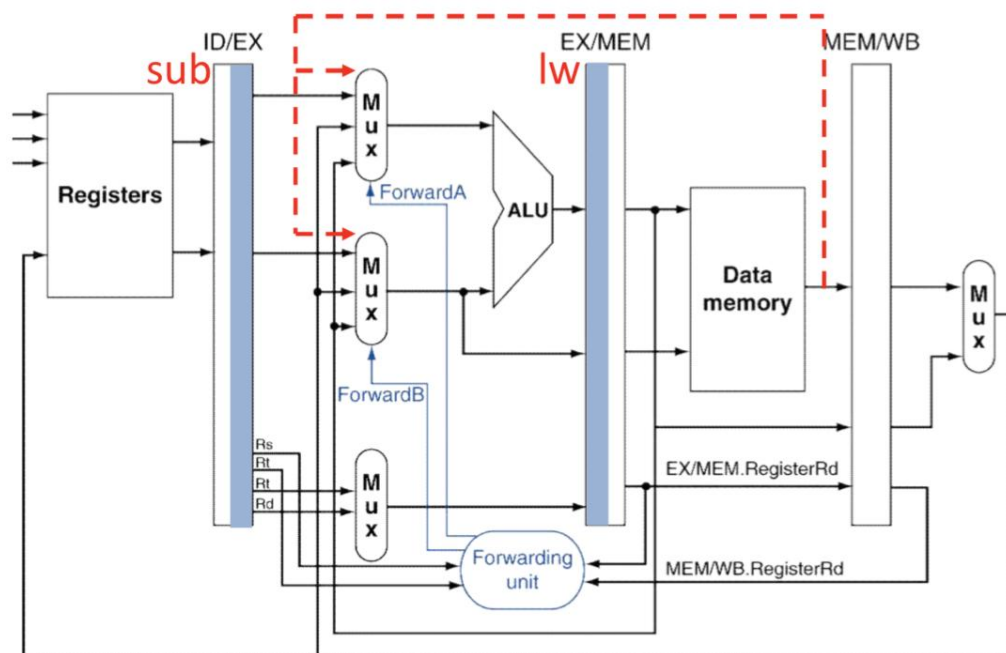
◆ 指令流

- lw: 结果保存至RF
- sub: 结果保存在EX/MEM

		IF级								ID级	EX级	MEM级	WB级
地址	指令	CLK	PC	IM	IF/ID	ID/EX	EX/MEM	MEM/WB	RF				
0	lw \$t0, 0(\$t1)	↑ 1	0→4	lw→sub	lw								
4	sub \$t3, \$t0, \$t2	↑ 2	4→8	sub→and	sub	lw							
8	and \$t5, \$t0, \$t4	↑ 3	8→8	and	sub	nop	lw						
12	or \$t7, \$t0, \$t6	↑ 4	8→12	and→or	and	sub	nop	lw结果					
16	add \$t1, \$t2, \$t3	↑ 5	12→16	or→add	or	and	sub结果	nop	lw结果				

数据冒险——阻塞

- Q: 如果设置从DM到ALU输入的转发, 这个设计优劣如何?
- 设计初衷: 将DM读出数据提前1个clock转发至ALU, 从而消除lw指令导致的数据相关, 无需插入NOP



数据冒险——阻塞

■ A: 功能虽然正确, 但CPU时钟频率大幅度降低

◆ 原设计: $f = 5\text{GHz}$

➢ 各阶段最大延迟为200ps

◆ 新设计: $f = 2.5\text{GHz}$

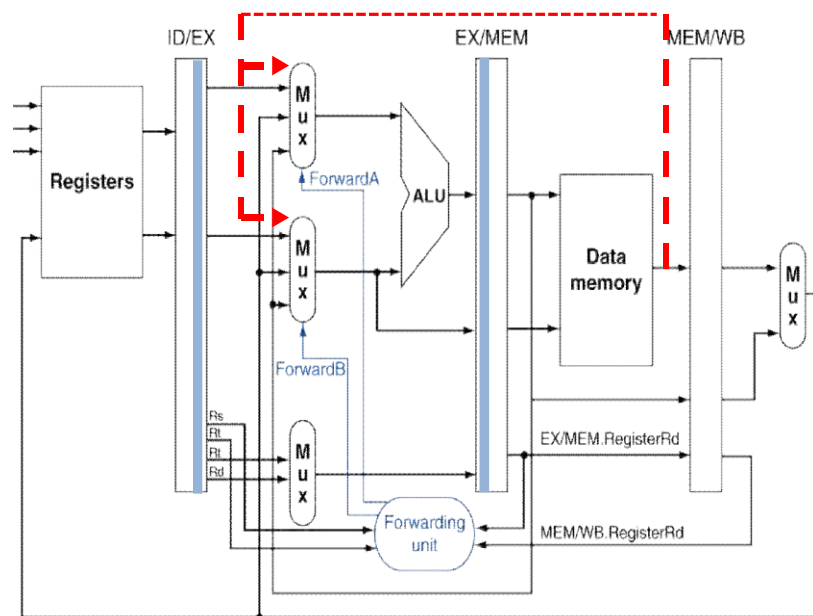
➢ 时钟长度: $\text{ALU延迟} + \text{DM延迟} = 400\text{ps}$

➢ EX阶段延迟成为最大延迟

警惕: 木桶原理!

流水线各阶段延迟不均衡,
将导致流水线性能严重下降

Instr fetch	Register read	ALU op	Memory access	Register write
200ps	100 ps	200ps	200ps	100 ps



数据冒险——阻塞

■ 如何插入NOP指令？

◆ 检测条件：IF/ID的前序是lw指令，并且lw的rt寄存器与IF/ID的rs或rt相同

◆ 执行动作：

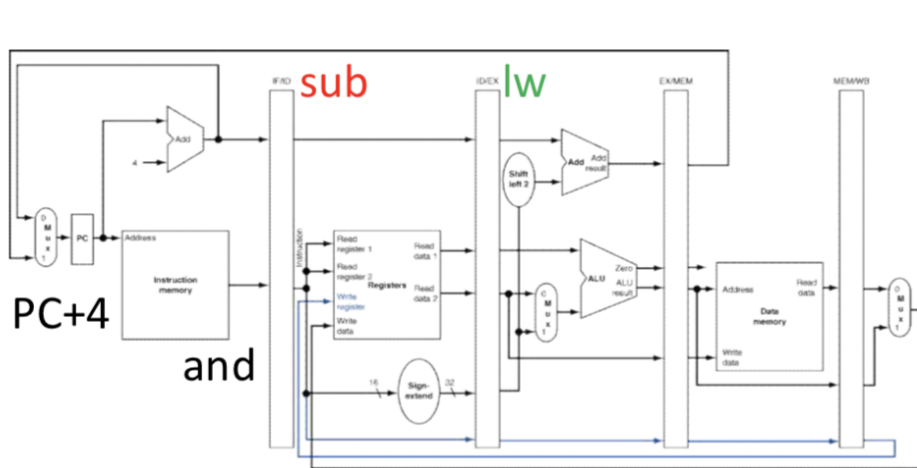
➢ 冻结IF/ID：sub继续被保存

➢ 清除ID/EX：控制指令全为0，等价于插入NOP

➢ 禁止PC：防止PC继续计数，PC应保持为PC+4

两个使能 + 一个 clear 控制信号

地址	指令
0	lw \$t0, 0(\$t1)
4	sub \$t3, \$t0, \$t2
8	and \$t5, \$t0, \$t4
12	or \$t7, \$t0, \$t6
16	add \$t1, \$t2, \$t3

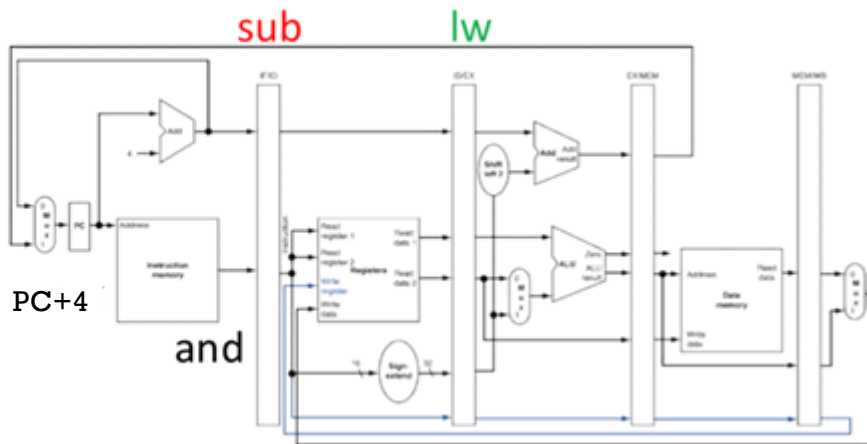


数据冒险——阻塞

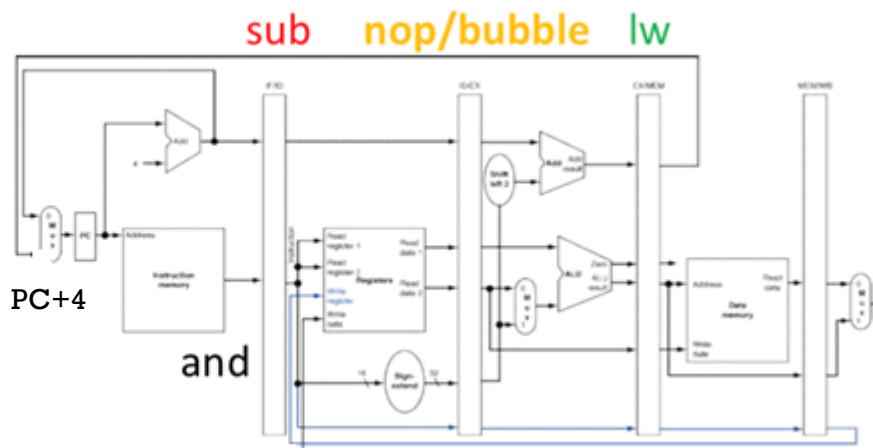
■如何插入NOP指令？

Cycle N

地址	指令
0	lw \$t0, 0(\$t1)
4	sub \$t3, \$t0, \$t2
8	and \$t5, \$t0, \$t4
12	or \$t7, \$t0, \$t6
16	add \$t1, \$t2, \$t3



Cycle N+1



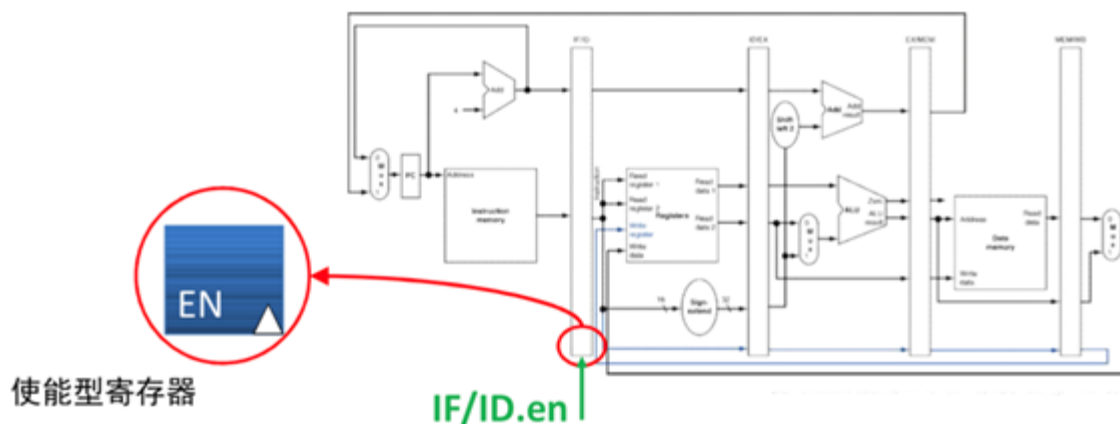
数据冒险——阻塞

■ 插入NOP指令，执行动作：

- ◆ 冻结IF/ID：sub继续被保存
- ◆ 清除ID/EX：指令全为0，等价于插入NOP
- ◆ 禁止PC：防止PC继续计数，PC应保持为PC+4

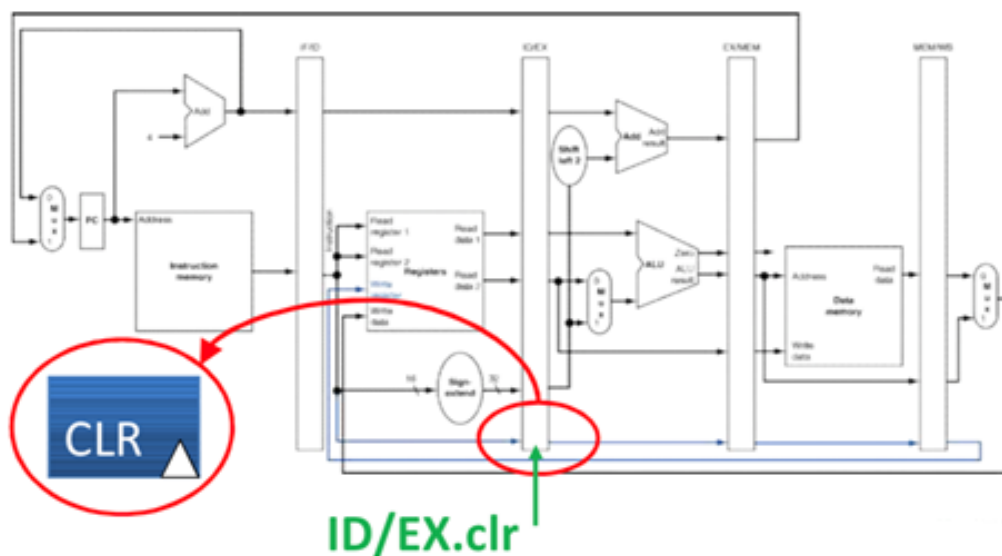
■ 数据通路：将IF/ID修改为使能型寄存器控制系统

- ◆ 增加IF/ID.en控制信号：当IF/ID.en为0时，IF/ID在下一个clock上升沿到来时保持不变



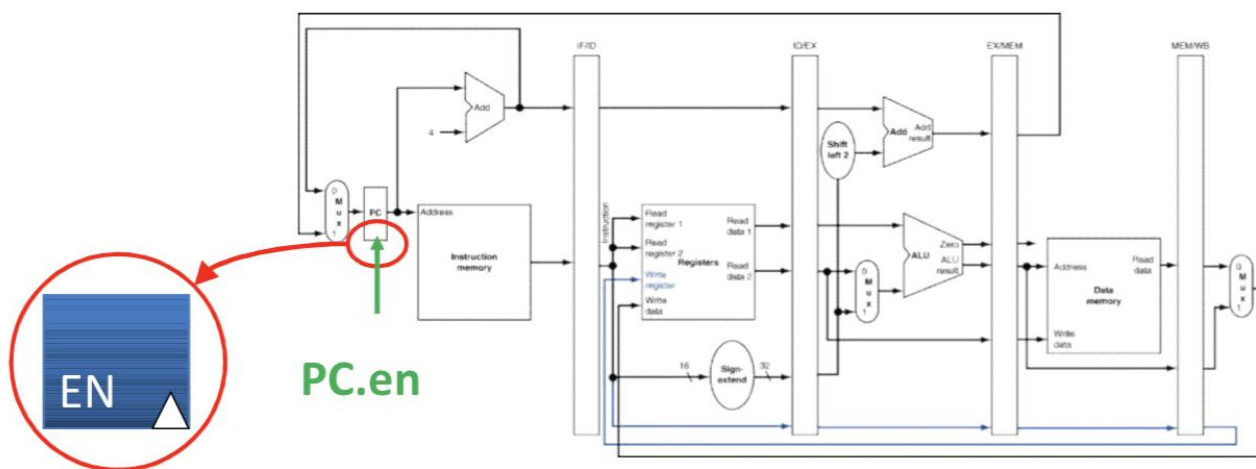
数据冒险——阻塞

- 插入NOP指令，执行动作：
 - ◆ 冻结IF/ID：sub继续被保存
 - ◆ 清除ID/EX：指令全为0，等价于插入NOP
 - ◆ 禁止PC：防止PC继续计数，PC应保持为PC+4
- 数据通路：将ID/EX修改为复位型寄存器控制系统
 - ◆ 增加ID/EX.clr控制信号：当ID/EX.clr为0时，ID/EX在下一个clock上升沿到来时被清除为0



数据冒险——阻塞

- 插入NOP指令，执行动作：
 - ◆ 冻结IF/ID: sub继续被保存
 - ◆ 清除ID/EX: 指令全为0, 等价于插入NOP
 - ◆ 禁止PC: 防止PC继续计数, PC应保持为PC+4
- 数据通路: 将PC修改为使能型寄存器 控制系统
 - ◆ 增加PC.en控制信号: 当PC.en为0时, PC在下一个clock上升沿到来时保持不变



数据冒险——阻塞

■ 如何插入NOP指令？

◆ lw冒险处理示例伪代码

◆ 注意：时序关系

- 各信号在clk2上升沿后有效
- NOP是在clk3上升沿后发生，即寄存器值在clk3上升沿到来时发生变化(或保持不变)

if(ID/EX.MemRead) & //前一条指令是lw
 ((ID/EX.rt == IF/ID.rs) |
 (ID/EX.rt == IF/ID.rt))
 IF/ID.en ← 禁止(disable)
 ID/EX.clr ← 清除(reset)
 PC.en ← 禁止(disable)

		IF级		ID级	EX级	MEM级	WB级		
地址	指令	CLK	PC	IM	IF/ID	ID/EX	EX/MEM	MEM/WB	RF
0	lw \$t0, 0(\$t1)	↑ 1	0→4	lw→sub	lw				
4	sub \$t3, \$t0, \$t2	↑ 2	4→8	sub→and	sub	lw			
8	and \$t5, \$t0, \$t4	↑ 3	8→8	and	sub	nop	lw		
12	or \$t7, \$t0, \$t6								
16	add \$t1, \$t2, \$t3								

数据冒险——阻塞

■ Q: 如果没有转发, 需要怎么处理呢?

■ A: EX/MEM, MEM/WB也均需要做冲突分析及NOP处理

◆ EX/MEM, MEM/WB也需要修改, 并增加相应控制信号

地址 指令

0 lw \$t0, 0(\$t1)

4 sub \$t3, \$t0, \$t2

8 and \$t5, \$t0, \$t4

12 or \$t7, \$t0, \$t6

16 add \$t1, \$t2, \$t3

Clk	PC	IM	IF	ID	EX	MEM	WB
↑ 1	0→4	lw→sub	lw				
↑ 2	4→8	sub→and	sub	lw			
↑ 3	8	and	and	sub	lw		
↑ 4	8	and	and	sub	nop	lw	
↑ 5	8→12	and→or	and	sub	nop	nop	lw结果
↑ 6	12→16	or	or	and	sub	nop	nop

数据冒险——阻塞

◆ 装载延迟槽：延迟与LW指令结果相关的指令执行

- 如果指令使用LW指令的结果，那么需要阻塞一个时钟周期
- 在延迟槽中阻塞一条指令等同于在槽中放入一个nop *底线方案*
- 更优方案：让编译器在延迟槽中放一个无关指令

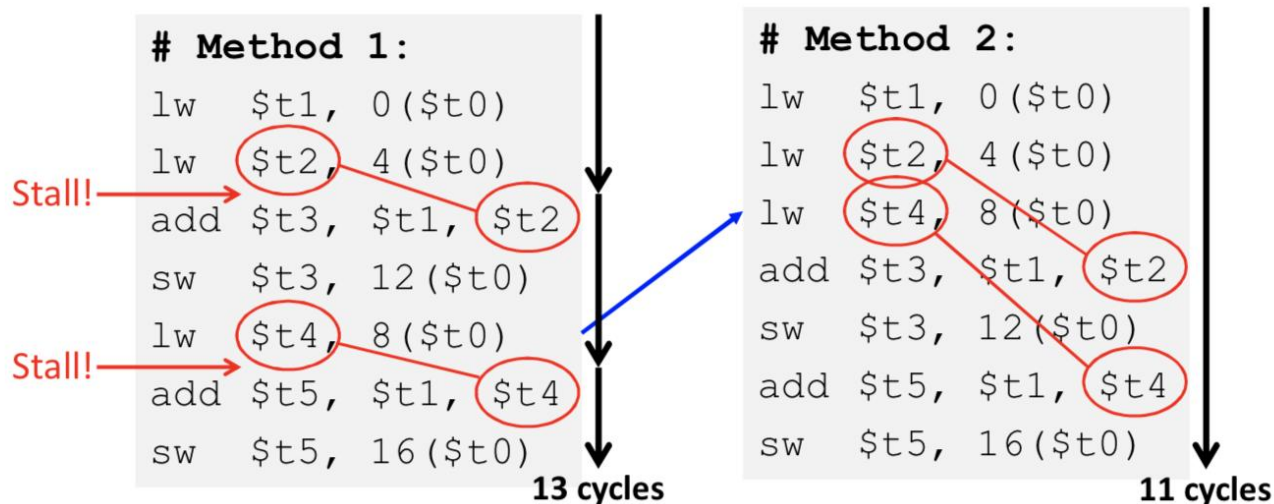
调整指令顺序

数据冒险——装载延迟槽

■ 重新排列代码，以避免流水线阻塞

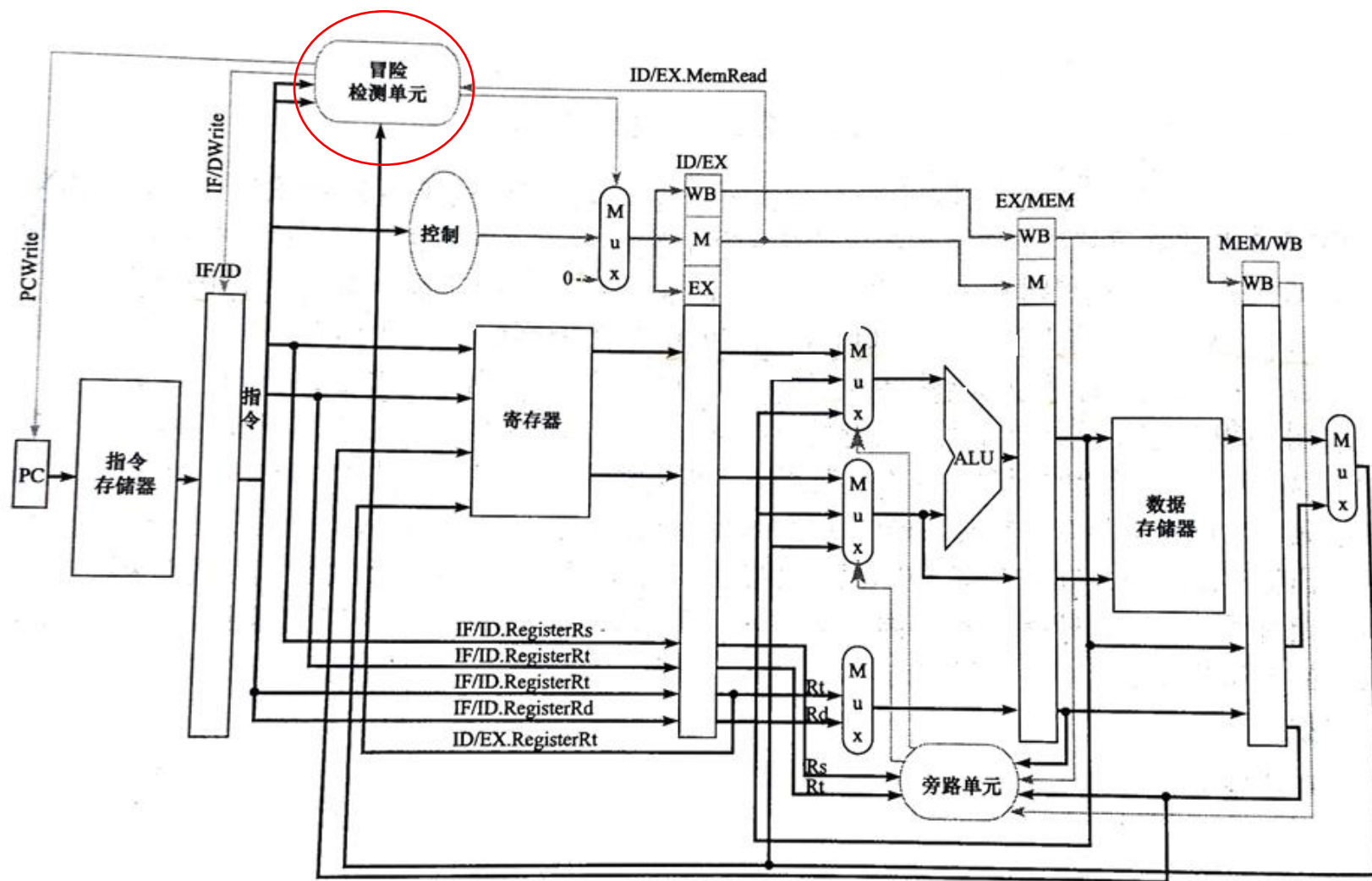
◆ 重新排列代码，防止在下个指令中使用装载指令结果。

◆ 考虑如下代码 $A = B + E$; $C = B + F$;



完美流水线N条指令总时钟周期数: $5 + (N - 1)$ 当N很大时 $CPI \approx 1$

■ 在控制单元、旁路单元外，再增加一个冒险检测单元



4.7.3 控制冒险（分支冒险）

- 控制冒险：为了确保预取正确指令而导致延迟
 - ◆ 出现频率比数据冒险小得多
- 分支 (beq, bne) 是控制的重要部分
 - ◆ 下次的取址取决于分支的结果
 - ◆ 流水线不一定总能取得正确的指令
- 简单的解决方案: 对每条分支指令进行阻塞，直到有一个新的PC
 - ◆ 需要阻塞多久？

控制冒险（分支冒险）

■ beq指令处理流程：beq rs, rt, imm16

■ If ($R[rs] - R[rt] = 0$) then $PC \leftarrow (PC + 4) + \text{Signext}(\text{imm16}) \ll 2$
else $PC \leftarrow PC + 4$

◆ IF

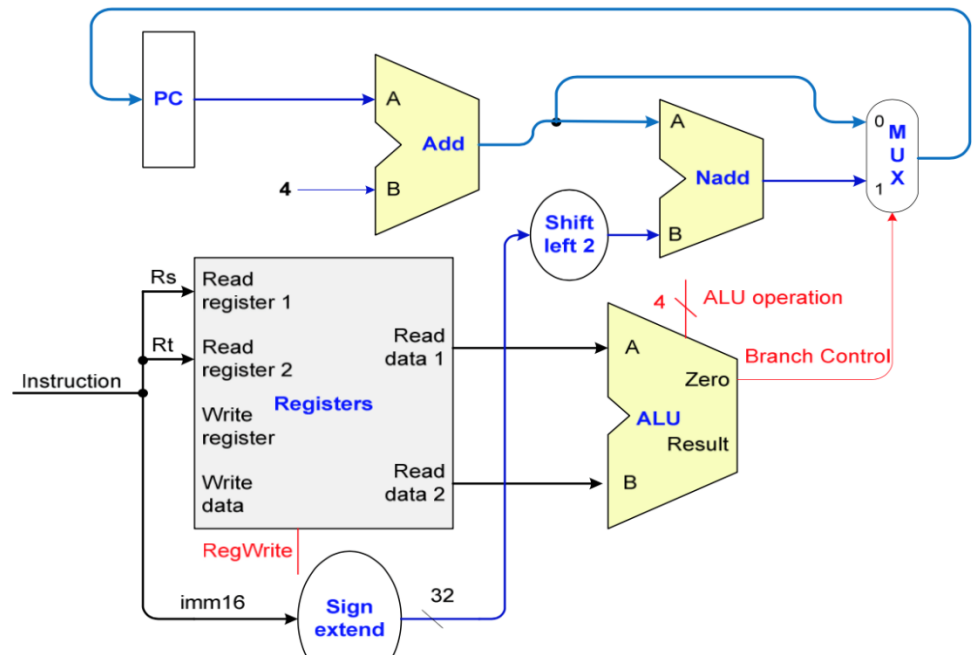
◆ ID：取 $R[rs]$, $R[rt]$

计算 $\text{signext}(\text{imm16})$

◆ EX：计算 $R[rs] - R[rt]$

计算 $PC + 4 + \text{signext}(\text{imm16}) \ll 2$

◆ MEM：根据比较结果决定PC选择



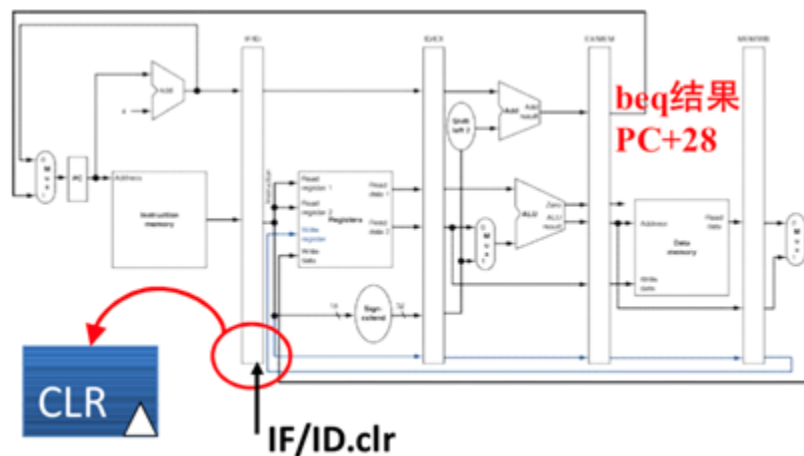
控制冒险（分支冒险）

■ 分支（Branch）指令冒险造成的停顿代价

地址	指令	CLK	PC	IM	IF	ID	EX	MEM	WB
0	beq \$1, \$3, 24	↑ 1	0→4	beq→and	beq				
4	and \$12, \$2, \$5	↑ 2	4	and	and	beq			
8	or \$13, \$6, \$2	↑ 3	4	and	and	nop	beq结果		
12	add \$14, \$2, \$2	↑ 4	4→28	and→lw	and	nop	nop	PC+Imm	
28	lw \$4, 50(\$7)	↑ 5	28→32	lw→XX	lw	nop	nop	nop	beq

□ 如不对B指令做任何处理，则必须插入3个NOP

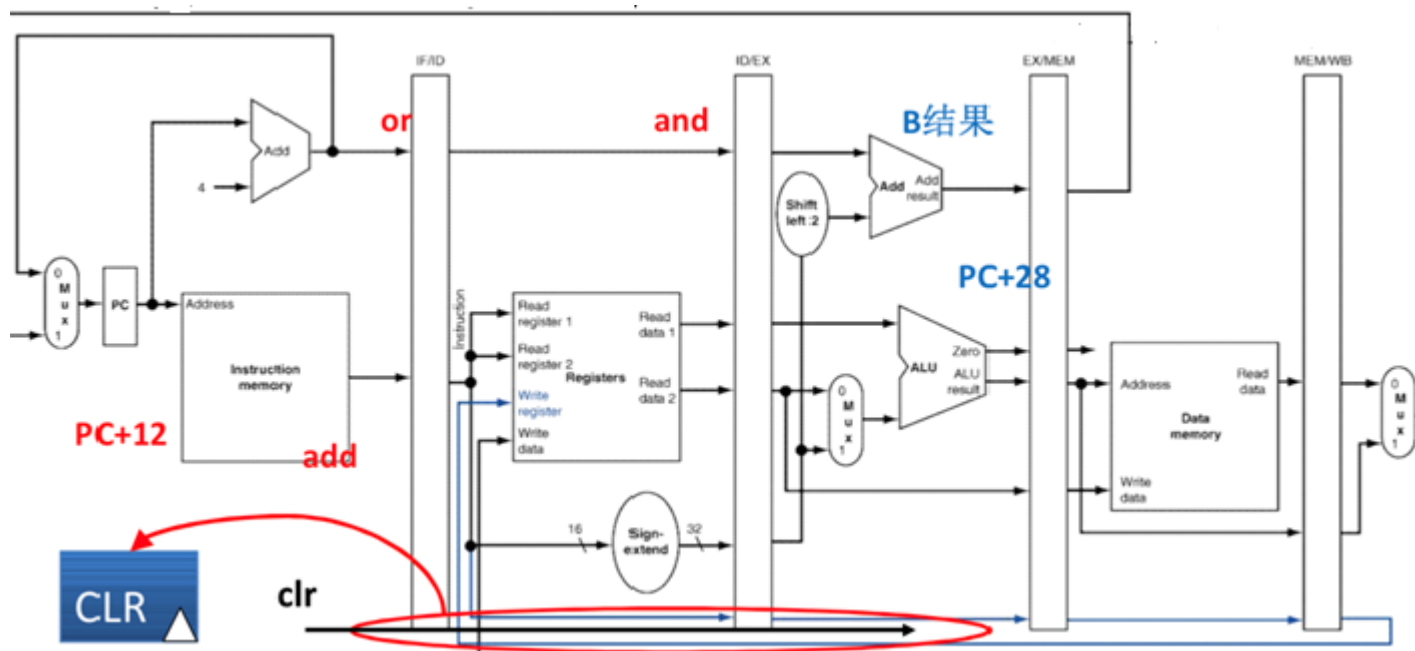
- ◆ b指令结果及新PC值保存在EX/MEM，因此PC在clk4才能加载正确值
- ◆ IF/ID在clk5才能存入转移后指令(即lw指令)



控制冒险（分支冒险）

- 方案1：正常执行，如分支发生则清除
 - ◆ 即使在ID级发现是B指令也不停顿
 - ◆ 根据B指令结果，决定是否清除3条后继指令
 - ◆ 清除流水线寄存器，使得and/or/add不能前进

PC相对偏移	指令
0	beq \$1, \$3, 24
4	and \$12, \$2, \$5
8	or \$13, \$6, \$2
12	add \$14, \$2, \$2
.....	
28	lw \$14,\$2,\$2



控制冒险（分支冒险）

■ 方案1的特点

- ◆ 相当于每次都预测条件不满足，不跳转，继续执行分支后续指令
- ◆ 若预测错误一次损失3个周期
- ◆ 静态预测，成功率不是特别高

控制冒险（分支冒险）

■ 方案2: 缩短延迟分支，提前计算分支结果

◆ 提前计算可能的 $PC+4+imm16 \ll 2$

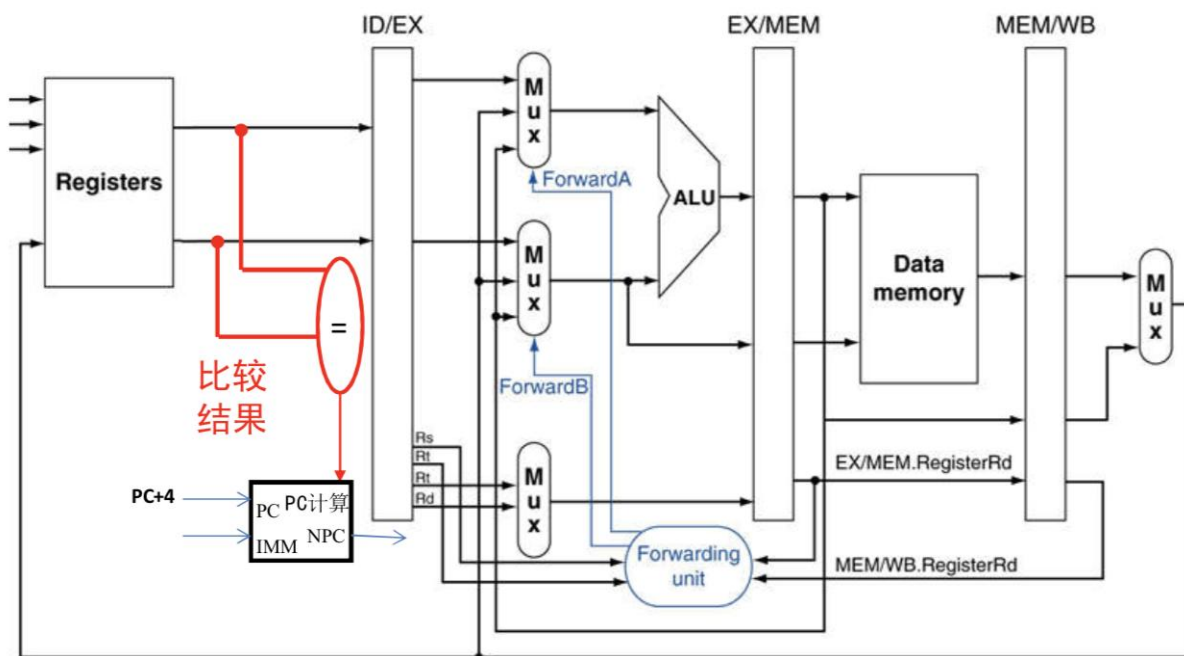
- IF/ID阶段已有PC和imm16
- 把计算从EX阶段提前到ID阶段即可

◆ 提前计算 $R[rs]-R[rt]$

- 在ID阶段从Reg中取出数据直接通过异或硬件判定相等
- 寄存器的数据依赖流水线中的结果，这种情况比较麻烦
 - 数据可从旁路获得：增加新的旁路检测单元
 - 需要冒险检测：可能还需要阻塞1个（rs或rt是前面ALU指令的目标）或2个周期（rs或rt是前面Lw指令的目标）

控制冒险（分支冒险）

- 方案2：缩短分支延迟，在ID阶段放置比较器，尽快得到B指令结果
 - ◆ B指令结果可以提前2个clock得到
 - ◆ B指令后继可能被废弃的指令减少为1条
 - 当需要转移时，清除IF/ID即可



控制冒险（分支冒险）

■ 方案2：缩短分支延迟

add \$1, \$2, \$3

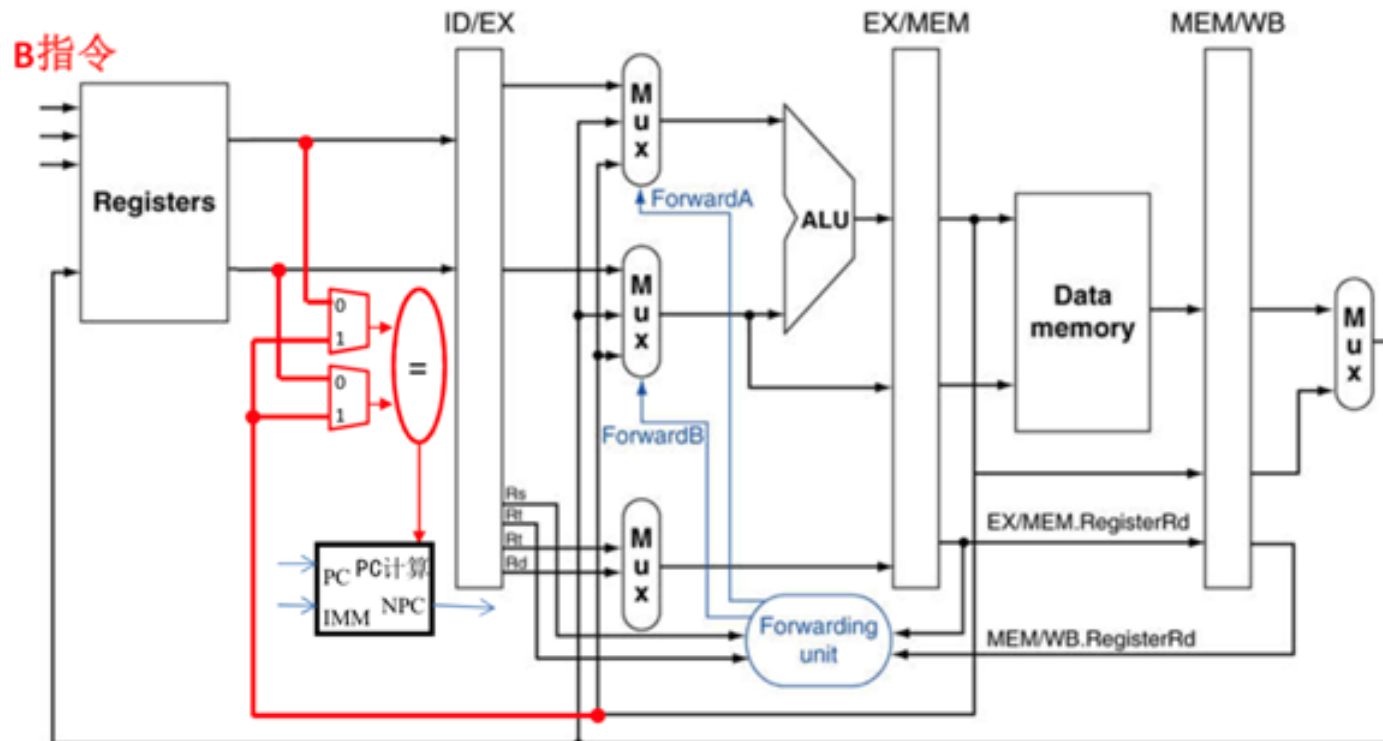
beq \$1, \$2, Exit

需要暂停吗？

lw \$1, 0(\$0)

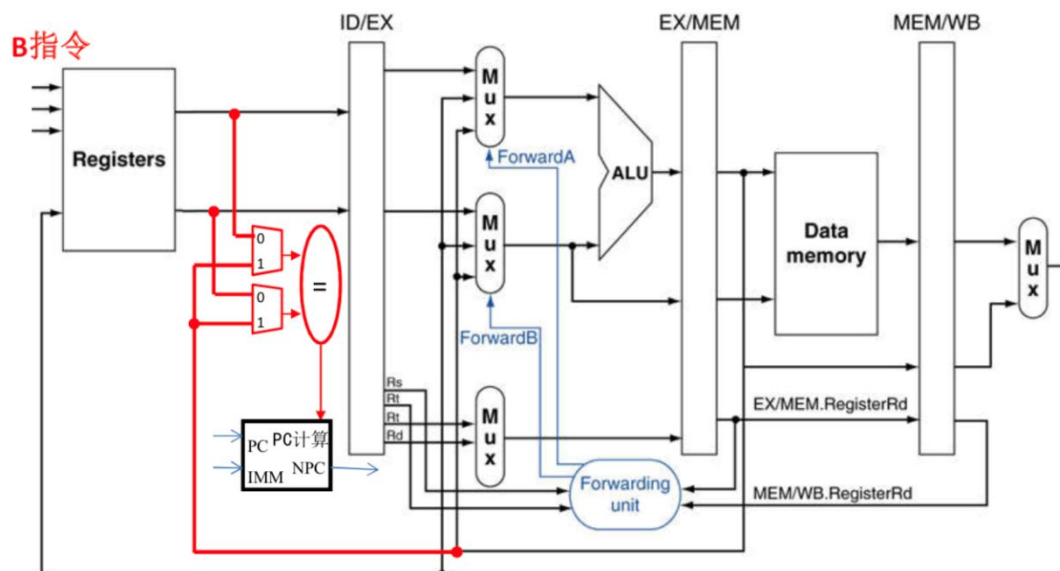
beq \$1, \$2, Exit

需要暂停吗？



控制冒险（分支冒险）

- Beq依赖前面一条指令结果，可能导致阻塞
 - ◆ 如果**beq**前面是ALU计算类型指令，阻塞1个周期
 - 旁路：EX/MEM -> ID
 - ◆ 如果**beq**前面是lw指令，阻塞2个周期
 - 旁路：MEM/WB -> ID



控制冒险（分支冒险）

■ 方案2的效果

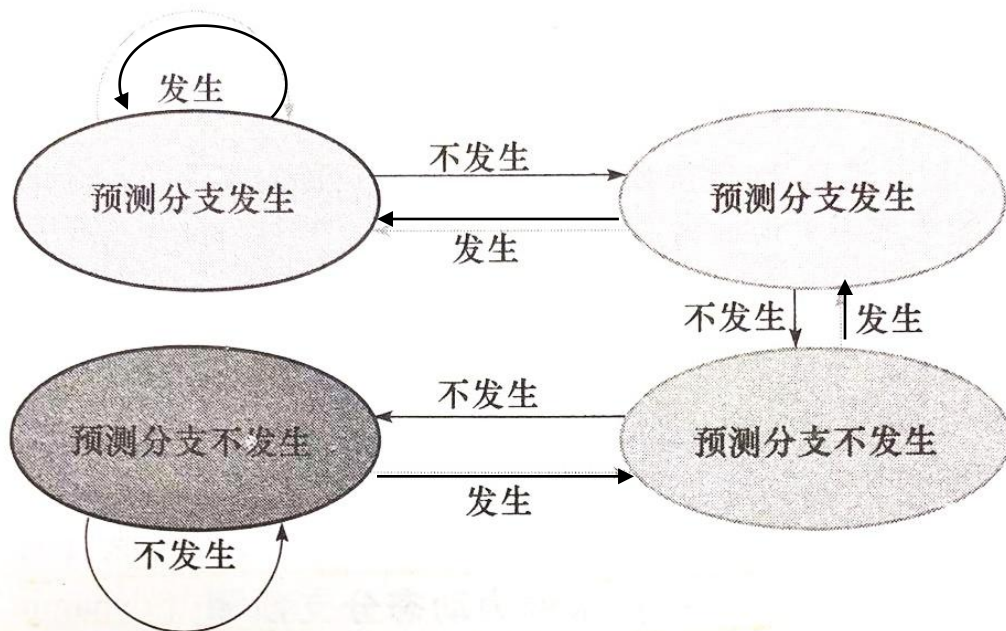
- ◆ 分支执行提前到ID阶段
- ◆ 分支预测错误的代价减小到最少可以只有一条指令

控制冒险（分支冒险）

■ 优化：动态分支预测

◆ 保存之前预测结果

◆ 之前判断正确，就继续；否则改变判断



控制冒险（分支冒险）

■ 方案2的进一步优化：分支延迟槽

- ◆ 分支延迟槽：位于分支指令后面的一条指令，不管分支发生与否，总是被执行。
- ◆ 最坏解决方案：在分支延迟槽中放入一个nop。
- ◆ 较好解决方案：在分支延迟槽放入分支指令前一个指令。
- ◆ 原则：不能影响程序的逻辑。

控制冒险（分支冒险）

■ 延迟分支举例



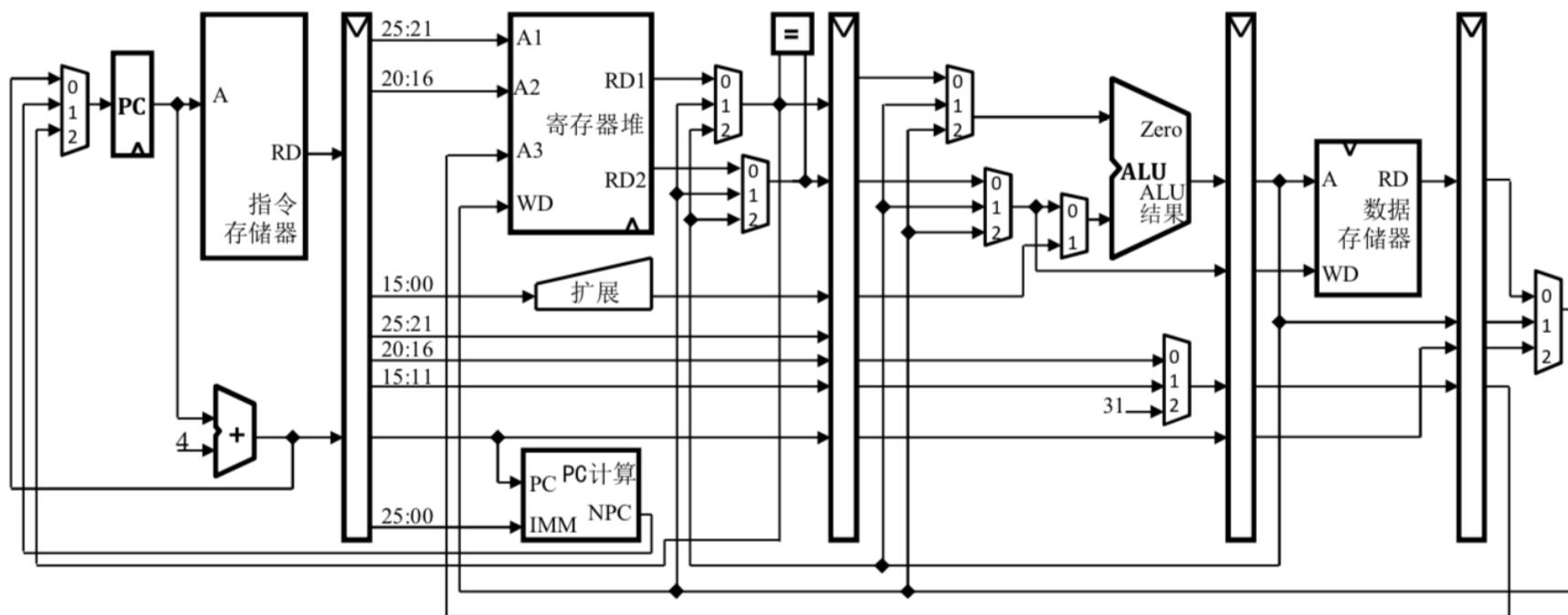
控制冒险（分支冒险）

■ MIPS使用延迟分支

- ◆ 重新排列指令使程序更优
- ◆ 编译器有50%的机会将一条指令放入分支延迟槽中（另外50%是nop)

控制冒险

- 跳转指令也需要一个延迟槽



控制冒险

■ MIPS中的延迟跳转

- ◆ jal:类似于call, 过程调用, 需要返回, 返回地址放在R[31]
- ◆ $R[31] = PC + 8$; $PC = \text{JumpAddr}$
 - $PC + 8$: 因为有延迟跳转槽
 - 位于 $PC + 4$ 的指令总会在跳转前执行, 所以返回位置 $PC + 8$

控制冒险

■ 延迟槽总结

- ◆ 范围：条件跳转 (beq, bne) , 无条件跳转 (j) , 过程调用 (jal)
- ◆ 硬件 (CPU) :
 - 静态处理方式
 - 知道实际的下一条指令是PC+8 (jal返回点) , PC+4是插入的无关指令或nop
- ◆ 软件 (Compiler) :
 - 尽可能将一条无关指令放入PC+4的位置 (50%)
 - 如果没有无关指令, 则放入nop (50%)

控制冒险

■延迟槽与分支预测

- ① 延迟槽+跳转提前到ID + beq/bne前没有数据依赖
 - 下一条指令取PC+4, 然后就可以确定下一条地址 (跳转/不跳转), 不用分支预测了
- ② 无延迟槽+跳转提前到ID + beq/bne前没有数据依赖
 - 只能先取PC+4 (相当于预测不调转, 因为跳转地址暂时还不知道)
- ③ beq/bne前面有数据依赖
 - Beq/bne跳转地址可以计算出来
 - 是否跳转的计算需要推迟 (因为有数据依赖), 这时需要分支预测, 尤其是动态分支预测 (准确率高), 让流水线不要停顿, 继续执行后面的指令